INDEX

NAME ADITHIYA.K SUBJECT GRAPHICS & MULTIMEDIA

STD. DIV. ROLL NO. SCHOOL

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
1	24/7/26	Bresenham Line Algorithm		
2	28/7/26	Bresenham Circle Algorithm		
3	29/7/26	Bresenham Ellipse Algorithm		
4	3/7/26	Draw a Butterfly		
5	2/8/26	Draw a smily face		
6	3/8/26	Flood-fill		
7	11/8/26	Olympic logo using ellipse		
8	20/8/26	Basic 2D transformation - Triangle		
9	21/8/26	Basic 2D transformation - Rectangle		
10	26/8/26	Sutherland Hodgman Polygon Clipping		
11	28/8/26	3D shape transformation - Cubes		
12	11/9/26	3D transformation - Cone		
13	15/9/26	Rendering 3D objects using polygons & Shading techniques		
14	22/9/26	Multimedia application integrating image, sound & video in VB		
15	1/10/26	Capture Video/Audio in Webcam or Microphone		

Completed

21/10/26

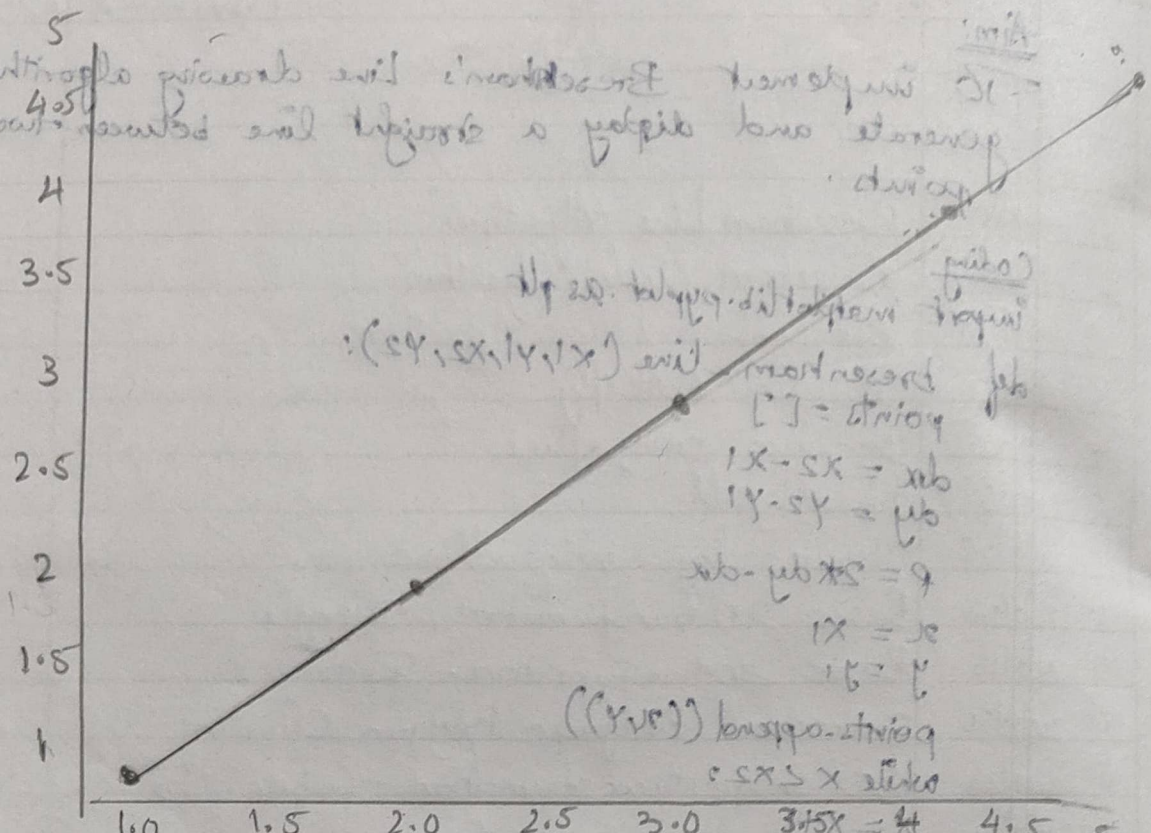
1-a BRESENHAM LINE DRAWING ALGORITHM

Aim:

To implement Bresenham's Line drawing algorithm to generate and display a straight line between two given points.

Coding:

```
import matplotlib.pyplot as plt
def bresenham_line(x1,y1,x2,y2):
    points = []
    dx = x2-x1
    dy = y2-y1
    p = 2*dy-dx
    x = x1
    y = y1
    points.append((x,y))
    while x<x2:
        x = x+1
        if p<0:
            p = p+2*dy
        else:
            y = y+1
            p = p+2*dy-2*dx
        points.append((x,y))
    return points
x1 = int(input("Enter x1: "))
y1 = int(input("Enter y1: "))
x2 = int(input("Enter x2: "))
y2 = int(input("Enter y2: "))
points = bresenham_line(x1,y1,x2,y2)
print("Points generated:")
for p in points:
    print(p)
x_values = [p[0] for p in points]
y_values = [p[1] for p in points]
plt.plot(x_values, y_values, marker="o")
plt.title("Bresenham Line Drawing Algorithm (|m|<1)")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid()
plt.show()
```


5
4.5
4
3.5
3
2.5
2
1.5
1
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5

31/7/26

Result:

Bresenham's line drawing algorithm has been implemented

1-b BRESENHAM'S MIDPOINT CIRCLE DRAWING ALGORITHM

Aim:

To implement midpoint circle algorithm to generate and display circle

Code:

```
import matplotlib.pyplot as plt
xc = int(input("Enter center x: "))
yc = int(input("Enter center y: "))
r = int(input("Enter radius: "))

x=0
y=r
p=1-r
x-points = []
y-points = []
while x<=y:
    points = [(xc+x,yc+y),(xc-x,yc+y),(xc+x,yc-y),
              (xc-x,yc-y),(xc+y,yc+x),(xc-y,yc+x),
              (xc+y,yc-x),(xc-y,yc-x)]

    for i in points:
        x-points.append(i[0])
        y-points.append(i[1])

    x+=1
    if p<0:
        p=p+2*x+1
    else:
        y-=1
        p=p+2*x-2*y+1

plt.scatter(x-points,y-points)
plt.title("midpoint circle")
```

```
plt.grid(True)
plt.axis("equal")
plt.show()
```

Result:

Bresenham's midpoint circle drawing algorithm has been implemented.

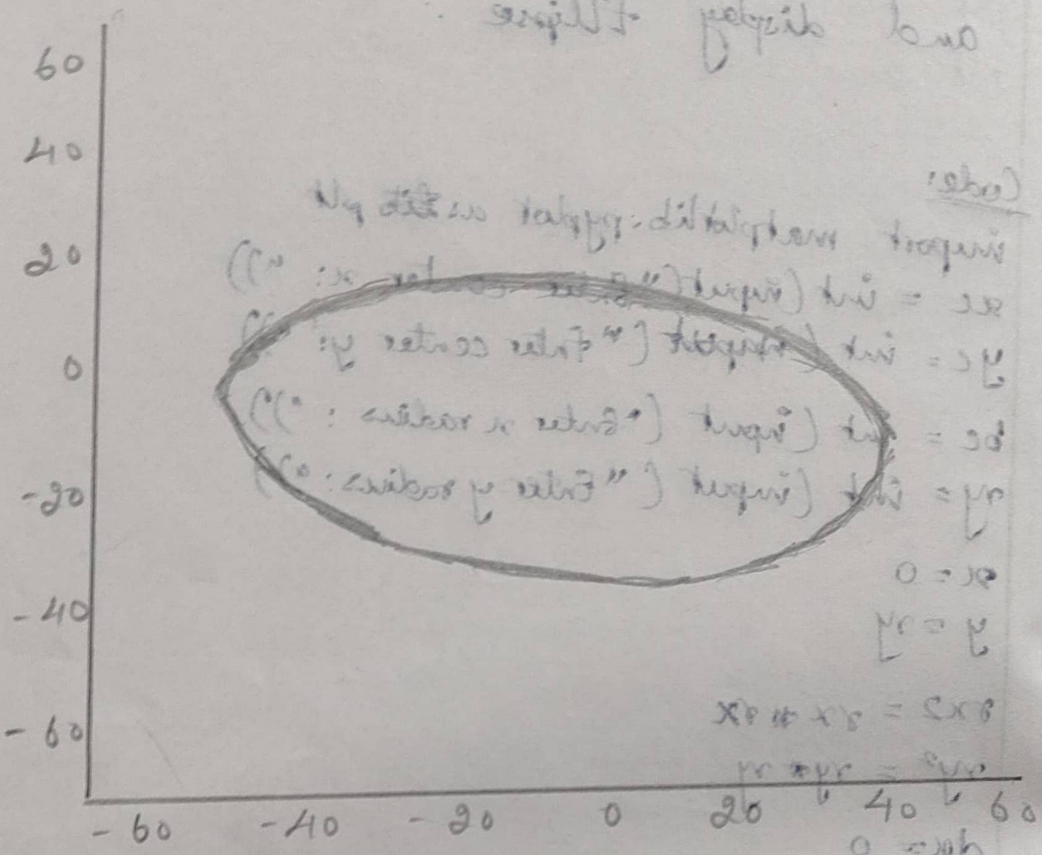
1-C - MIDPOINT ELLIPSE ALGORITHM

Aim:

To implement midpoint Ellipse Algorithm to generate and display Ellipse.

Code:

```
import matplotlib.pyplot as plt
xc = int(input("Enter center x: "))
yc = int(input("Enter center y: "))
rx = int(input("Enter x radius : "))
ry = int(input("Enter y radius: "))
x = 0
y = ry
rx2 = rx * rx
ry2 = ry * ry
dx = 0
dy = 2 * rx2 * y
p1 = ry2 - rx2 * ry + 0.25 * rx2
x_points = []
y_points = []
while dx < dy:
    points = [(xc+x, yc+y), (xc-x, yc+y), (xc+x, yc-y),
              (xc-x, yc-y)]
    for i in points:
        x_points.append(i[0])
        y_points.append(i[1])
    x += 1
    dx += 2 * ry2
```


60
40
20
0
-20
-40
-60
-60
-40
-20
0
20
40
60

```
if p1<0:
    p1+= dx+ry**2
else:
    y-=1
    dy -= 2*rx**2
    p1+= dx-dy+ry**2

p2=ry**2*(x+0.5)**2+rx**2*(y-1)**2-rx**2*ry**2
while y>=0:
    points = [(xc+x, yc+y), (xc-x, yc+y), (xc+x, yc-y),
              (xc-x, yc-y)]
    for i in points:
        x-points.append(i[0])
        y-points.append(i[1])

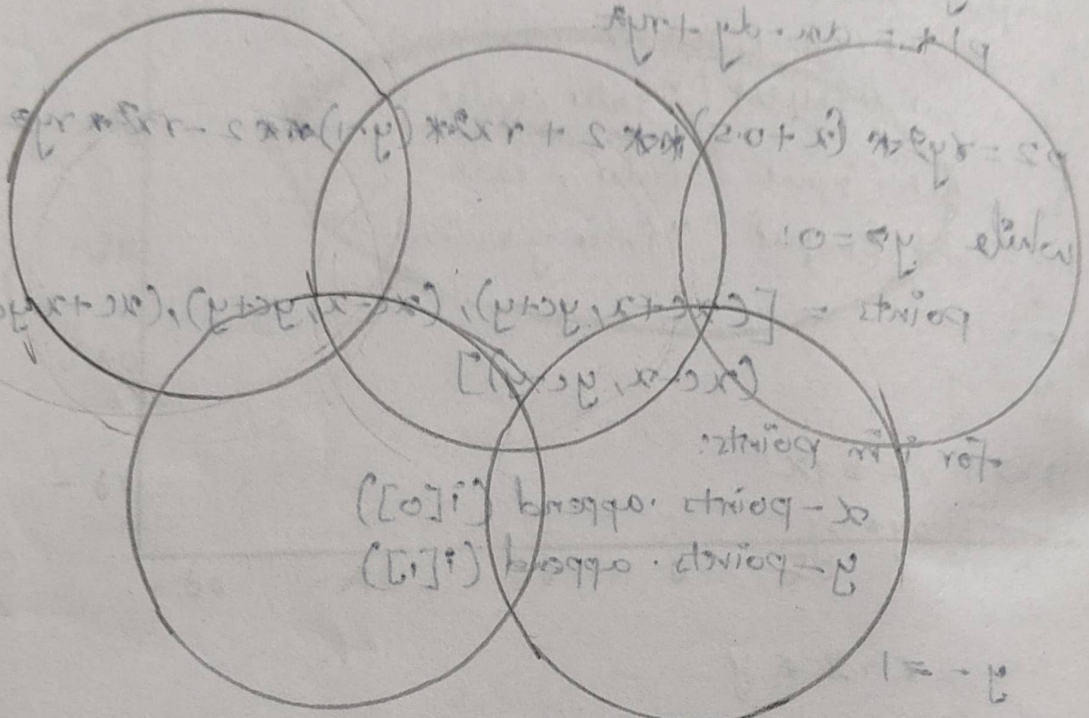
    y-=1
    dy-= 2*rx**2
    if p2>0:
        p2+= dx-dy+rx2

plt.scatter(x-points, y-points)
plt.title("midpoint ellipse")
plt.grid(True)
plt.axis("equal")
plt.show()
```

Result:

The Midpoint Ellipse Algorithm has been implemented.

Olympic Logo



2(a)

DRAW THE OLYMPIC LOGO USING ELLIPSE (MID POINT ALGORITHM)

AIM:

To draw the olympic logo using ellipse & midpoint algorithm

CODE:

```
import matplotlib.pyplot as plt
def midpoint_circle(cx,cy,r):
    point = []
    x = 0
    y = r
    p = 1-r
    while x<=y:
        points.extend([
            (cx+x, cy+y),
            (cx-x, cy+y),
            (cx+x, cy-y),
            (cx-x, cy-y),
            (cx+y, cy+x),
            (cx-y, cy+x),
            (cx+y, cy-x),
            (cx-y, cy-x)
        ])
    if p<0:
        p = p+2*x+3
    else:
        p = p+2*(x-y)+5
        y-=1
    x+=1
    return points
```

```
def draw_circle (cx, cy, r):
    points = midpoint_circle (cx, cy, r)
    points = sort(key= lambda p:--- import---
        ('math').atan2(p[1]-cy, p[0]-cx))

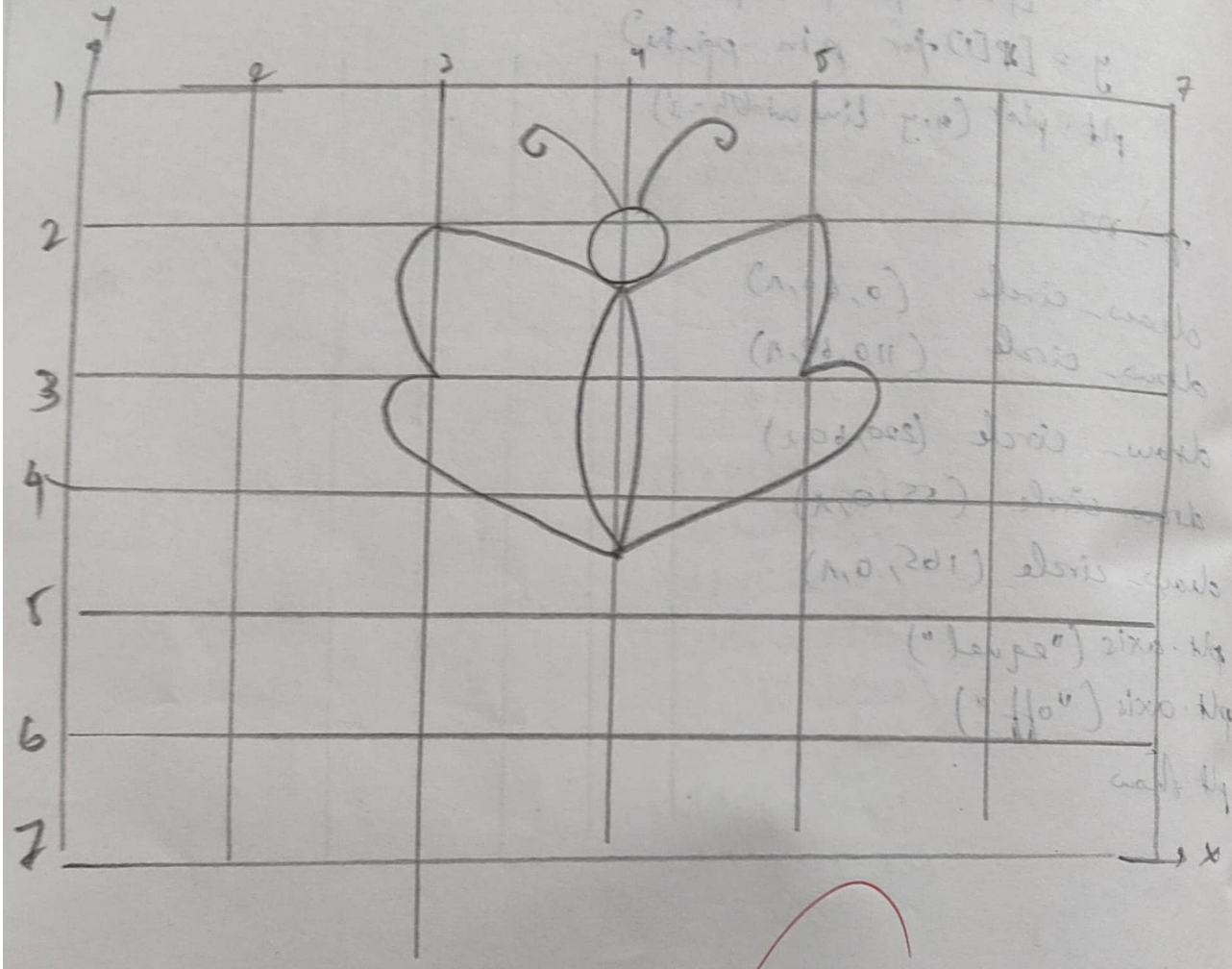
    x = [p[0] for p in points]
    y = [p[1] for p in points]
    plt.plot (x,y linewidth=3)

r = 80
draw_circle (0, 60, 1)
draw_circle (110, 60, 1)
draw_circle (220, 60, 1)
draw_circle (55, 0, 1)
draw_circle (165, 0, 1)
plt.axis ("equal")
plt.axis ("off")
plt.show
```

RESULT

The olympic logo using midpoint algorithm is drawn successfully.

Butterfly using Midpoint



2(b) DRAW BUTTERFLY USING MIDPOINT ALGORITHM

Aim:

To draw a butterfly using midpoint algorithm

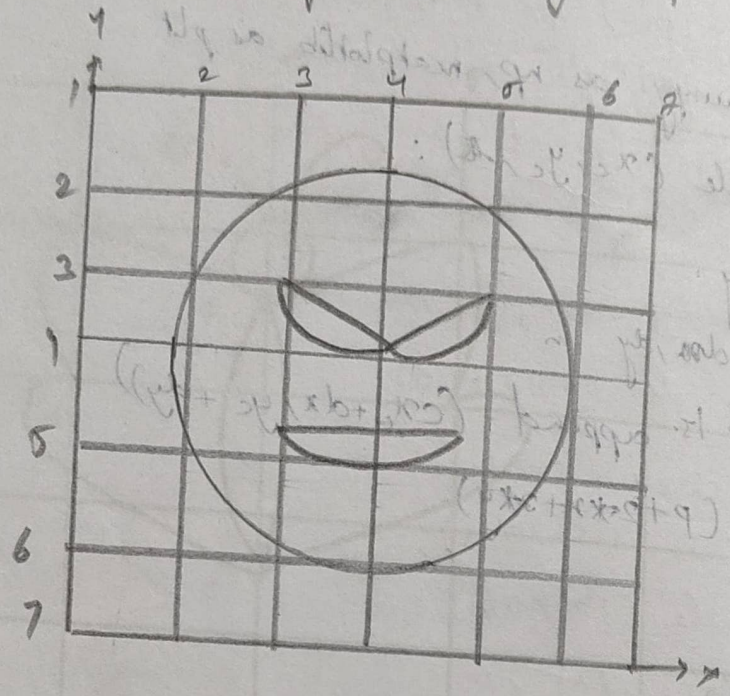
Code:

```
import numpy as np, matplotlib as plt
def get_circle (xc, yc, r):
while x<=y:
    for dx, dy in
        p is. append (xc+dx, yc+dy)
    p.y = (p+2*x+3*y)
    x+=1
return pts
    def plot_fill (pts, xc, yc, color)
    plt = sorted (set (pts), key=lambda
                  p: np.arctan2 (p[1]-yc, p[0]-xc))
    plt.figure (figsize=(6,6))
    plot_fill (get_ellipse (2,6,4,3) 'hot pink')
    plot_fill (get_circle (5,6,1) 'black')
    plot_fill (get_circle (5,9,4) 'black')
    plt.axis ('equal')
    plt.grid (True)
    plt.show
```

RESULT

Thus, a butterfly is drawn successfully.

Smiley Face using Midpoint



2(c)

DRAW SMILEY FACE USING MIDPOINT ALGORITHM

Aim:

To draw a smiley face using mid point circle algorithm

Code:

```
import matplotlib.pyplot as plt
import numpy as np
def mid_point_circle(xc, yc, r):
    points = []
    x = 0
    y = r
    p = 1 - r
    while x <= y:
        points.extend([
            (xc + x, yc + y),
            (xc - x, yc + y),
            (xc + x, yc - y),
            (xc - x, yc - y),
            (xc + y, yc + x),
            (xc - y, yc + x),
            (xc + y, yc - x),
            (xc - y, yc - x)
        ])
        if p < 0:
            p += 2 * x + 3
        else:
            p += 2 * (x - y) + 5
            y -= 1
        x += 1
    return points
```

```
def filled_circle (xc,yc,r, color)
    y= np. arange (yc-r, yc+r, 0.01)
    for yy in y:
        half = np.sqrt (r*r - (yy.yc)**2)
        plt.plot (
            [xc_half, xc+half],
            [yy,yy],
            color = color
        )

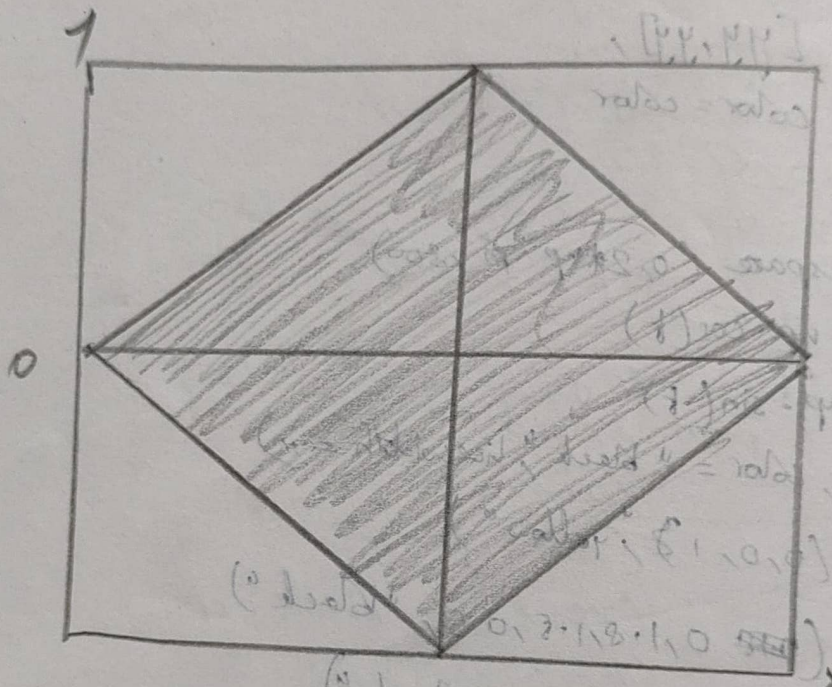
t = np. linespace (0, 2*np.pi, 800)
x = xc + r* np.cos(t)
y = yc + r* np.sin(t)
plt.plot (x, y, color = "black", linewidth = 2)
filled_circle [0,0, 1]; "yellow")
filled_circle (0, 1.8, 1.5, 0.5, "black")
filled_circle (1.8, 1.5, 0.5, "black")
t = np.linespace (np.pi + 0.2, 2*np.pi - 0.2, 150)
x = 2.5* np.cos(t)
y = 1.5* np.sin(t) - 0.5
plt.show()
```



RESULT

To draw smiley face using midpoint algorithm is done successfully.

Polygon Filling



FLOOD FILL ALGORITHM

Aim:

To draw a polygon using flood fill algorithm

CODE:

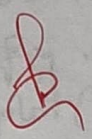
```
import matplotlib.pyplot as plt
import numpy as plt
width = 300
height = 300
image = np.ones (height, width, 3) dtype = np
                (unit 8) * 255
polygon = [(150,50), (250,130),
           (210,340), (90,240),
           (80,180)]

def draw_line (x1, y1, x2, y2):
    points = max (abs (x2-x1), abs (y2-y1))
    for i in range (points+1):
        x = int (x1 + (x2-x2) * i/points)
        y = int (y1 + (y2-y1) * i/points)
        image [y,x] = [0,0,0]
for i in range (len(polygon)):
    x1, y1 = polygon [i]
    x2, y2 = polygon [(i+1) % len (polygon)]
    draw_line (x1, y1, x2, y2)

def flood_fill (x,y, old_color, new_color):
    stack [(x,y)]
```

```
while stack:
    x, y = stack.pop()
    if x<0 or x>=width or y<0 or
            y>=higher continue
    if not np.array_equal(image
                [y,x], old_color):
            continue
    image [y,x] = new_color
    stack.append((x+1,y))
    stack.append((x-1,y))
    stack.append((x,y+1))
    stack.append((x,y-1))

flood_fill(
        180,
        180,
        np.array([255,255,255], dtype=np.uint8),
)
plt.imshow(image)
plt.title("Flood fill algorithm")
plt.axis("off")
plt.show()
```



RESULT

A polygon is drawn using the floodfill algorithm successfully.

BASIC TRANSFORMATION FOR 2D SHAPES FOR TRIANGLE

AIM: To implement the program for the basic transformation for 2D shapes to draw triangles in Python

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
shape = np.array([[0,0], [2,0], [1,2], [0,0]])
def plot(ax, s, color, label):
    ax.plot [s [:,0], s[i,1]. color=color, marker='o', label)
    ax.grid ()
    ax.axis('equal')
    ax.legend ()
def translate(s, dx, dy):
    return s+[dx, dy]
def rotate (s, angle):
    a = np.radians(angle), R = np.array
        ([[np.cos(a), -np.sin(a)],
          [np.sin(a), np.cos[a]]])
    return s@ R, T
def scale (s, sx, sy):
    return s = [sx, sy]
i) t = translate (shape, 2, 1)
   fig, ax = plt.subplot()
   plot(ax, shape, 'b/w', 'original'),
   plot (ax, t, 'red', 'trandate')
   ax.set_title ('2D translation')
   plt.show()
```

ii) r = rotate (shape, 4)
fig, ax = plt.subplots()
plot (ax, r, 'blue', 'original'), plot (ax, 1, 'green'; 'rotated')
ax.set_title ('2D rotation)
plt.show()

iii) s = scale (shape, 2, 0.))
fig, ax = plt.subplots')
plot (ax, shape, 'blue', 'original'), plot
(ax, s, 'purple', 'scale')
ax.set_title ('2D scaling')
plt.show()

RESULT:

Thus, a triangle is drawn for basic transformation in python

BASIC TRANSFORMATIONS FOR 2D SHAPES FOR RECTANGLE

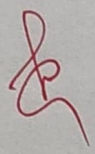
AIM:

To implemat the basic transformations for 2D shapes to draw rectangle in python

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
def translate (s, dx, dy):
    return s+[dx, dy]
def rotate (s, angle):
    a = np.radians (angle)
    return s@np.array ([R]).T
def scale (s, sx, sy):
    return s* [sx, sy]
def plot (ax, s, color, label):
    ax.plot (s[:, 0], s[:, 1], color = color, label = label)
rect = np.array ([[0, 0], [3, 0], [3, 2], [0, 4], [0, 0]])
fig, ax = plt.subplots ()
plot (ax, rect, 'blue', 'original'),
plot (ax, b, 'red', 'translation')
ax.set_title ('2D Translation')
ax.grid() ax.axis ('equal' ax.legend()
plt.show()
r = rotate (rect, 30
fig, ax = plt.subplots()
plot (ax, rect, 'blue', 'original'),
plot (ax, r, 'green', 'rotation').
```

```
ax.set_title('2D rotation')
ax.grid() ax.axis('equal') ax.legend()
plt.show()
s = state(rect, 0.5, 1.5)
fig, ax = plt.subplots()
plot(ax, rect, 'blue', 'original'),
plot(ax, s, 'purple', 'scaling')
ax.set_title('2D scaling')
ax.grid() ax.axis('equal') ax.legend()
plt.show()
```



RESULT:

The program to draw the rectangle for the basic tranformation in python is executed successfully.

SUTHERLAND-HODGMAN POLYGON CLIPPING ALGORITHM

Aim:

To implement the algorithm to clip a subject polygon against a rectangular clipping window using python.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt

def inside (p, a, b):
    return np.cross(b-a, p-a) <= 0

def intersect (p1, p2, a, b):
    d1 = p2 - p1
    d2 = b - a
    c = np.cross(a-p1, d2) / np.cross(d1, d2)
    return p2 + c * d1

def clip (subject, window):
    output_list (subject)
for i in range (len(window)) - 1):
    a, b = window [i], window [i+1]
    i, p - output = output, []

for j in range (len(inp)):
    p1, p2 = inp [j-1], inp [j]
        in1, in2 = inside [p1, a, b),
            inside (p2, a, b)

if in2:
    if not in1:
```

```
        output.append (intersect (p1,p2,a,b))

    elif i+1:
        output.append (intersect (p1, p2, a, b))

return np.array (output)
subject = np.array [[[0.5, 0.5], [3, 0.5], [2,3], [0.5, 2], [0.3, 0.5]])
window = np.array [[[1,1], [4,1], [4,4], [1,4], [1,1]])
result = clip (subject, window)
plt.plot (subject ~~window~~ [i,0], subject [i,1],
          'b-o', labels = 'subject')

if len (result) > 0:
    result = np.vstack [[result, result [0]]])
plt.title ('polygon clipping').plt.xlabel ('nc').plt.ylabel('y')

plt.legend ()
plt.show ()
```

RESULT:

The program for the sutherland hodgmen polygon clipping algorithm is done

3D SHAPES TRANSFORMATIONS FOR CUBE

AIM:

To implement the transformations for 3D shapes for cube using the python

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
v = np.array ([[0,0,0] ,[1,0,0], [0,1,0][0,0,1],[1,1,0], [1,0,1], [0,1,1],
               [1,1,1]])
M = np.c_[v, np.ones (8)]
F = [[0,1,4,2], [3,5,7,6] [0,1,5,3) [2,7,7,6)
     [0, 2, 6,3]  [1,4,7,5]]

def plot (T, title, color):
    w = (T @ M.T, T[:,:3]
    fig = plt.figure (figsize = (8,6))
    ax = fig.add_subplot (111, projection = '3d')
for v,c, in [(v, 'blue') (w,color)]
    for i,j in E:
        ax.plot (zip (v[i][i]) color = c)
        ax.scatter(v[i,0] , v[i,1] ,v[i 2], color = c)
faces = [[v[k] for R  in f] for f in t]
plt.show()

T = np.array [[1,0,0,2], [0,1,0,2], [0,0,1,2] [0,0,0,1]])
a = np.pi/4
T = np.dirg ([1,5,0,5,2,1] plot[t]
```

RESULT:

The program for the 3D tranformation for work is done.

3D TRANSFORMATIONS FOR CONE

Aim:

To implement the program for the 3D shapes tranformation for cone

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
v = np.array [[[0,0,0], [1,0,0], [1,1,6], [0,1,0], [15,15,1]]]

H = np.c_[v, np.ones(5)]
E = [(0,1), (1,2), (2,3), (3,6), (0,4), (1,4), (2,4), (3,4)]
F = [[0,1,2,3], [0,1,4], [1,2,4], [2,3,4], [2,3,4], [3,6,4]]

def plot(T, title, color):
    w = (T@H.T.T)[:,:3]
    fig = plt.figure(figsize=(8,6))
    ax = fig.add_subplot(111, projection='3d')

for T, c in [(v, 'blue'), (w, color)]:
    for i,G in E:
        ax.add_collections3d(Poly3DCollection([[p[k]
                                for k in f]
                                for k in f]
                                for f in k], alpha=.2, facecolor=c))
    ax.set(xlabel='x', ylabel='y', zlabel='z',
           title=title())
    plot = show()

T = np.array([[1,0,0,2], [0,1,0,2], [0,0,1,2]
    a = np.pi/4
    T = np.array([[np.cos(a), np.sin(a), 0, 0],
                  [np.sin(a), np.cos(a), 0, 0], [0,0,1,0],
                  [0,0,0,1]])
    T = np.diag([1,3,3,2,1])
```

RESULT

The program for 3D transformations for cone is implemented.

RENDERING 3D OBJECTS USING POLYGONS & SHADING TECHNIQUES

AIM:

To make & render 3D objects using polygonal modelling & apply flat shading & basic color techniques

PROGRAM:

```
import matplotlib.pyplot as plt
from mpl_toolkit.mplot3D import poly3Dcollection
import numpy as np
vertices = np.array([[0,0,0],[1,0,0],[1,1,2],[0,1,0],
                     [0,0,1],[1,0,1],[1,1,1],[0,1,1]])

faces = [[vertices[i] for j in [0,1,2,3]],
         [vertices[i] for j in [4,5,6,7]],
         [vertices[i] for j in [0,1,5,4]]]
colors = ['red', 'blue', 'green', 'yellow', 'orange', 'purple']
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
cube = poly3Dcollection(faces, facecolors=colors, alpha=0.7)
ax.scatter(vertices[:,0], vertices[:,1], vertices[:,2], color='black')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.set_xlim(0,1)
ax.set_ylim(0,1)
plt.show()
```

RESULT:

3D polygonal objects were successfully created and rendered.

MULTIMEDIA APPLICATION INTEGRATES IMAGES, SOUND AND VIDEO IN UI

AIM:

To develop a simple multimedia application that integrates images, video and audio in UI.

PROGRAM:

```
import tkinter as tk
from PIL import image, imagetk
from playground import playground
import cv2
import threading
def image():
    img_image ('sample_image.jpg').resize((300,300)
    photo = imagetk.photoImage(img)
    panel.config(image=photo)
    panel.image= photo
def audio():
    playsound("sample_audio.mp3")
def video():
    cap = cv2.VideoCapture("sample_video.mp4")
    while cap.isOpened():
        if not ret:
            break
        if(cv2.waitKey(25) & 0xFF== ord('q'):
        break
    cap.release()
    cv2.destroyAll((window)
    act.title("Multiplication app")
    panel.pack()
proof.mainloop()
```

RESULT:

A multimedia application integrating images, sound and video into a UI.

CAPTURE VIDEO/AUDIO FROM WEBCAM OR MICROPHONE AND DISPLAY IN MULTIMEDIA

AIM:

To develop a program that captures live video, audio from the webcam and microphone

PROGRAM:

```
import tkinter as tk
import cv2
import sounddevice as sd
from scipy.io.wavfile import write
  import threading
def audio():
    fs = 44100
      print("Recording ....")
    data = sd.rec(c = fs, samplerate = fs, channels = 2)
    sd.wait()
    write(".recorded_audio.wave", fs, data)
    print("Audio saved!")
def video():
      cap = cv, videocapture(0)
      while True:
        ret = frame = (cap.read()
          if not ret:
            break
        cv2 = imshow("webcam", frame)
if cv2, waiting(1) & 0XFF == ord('q'):
break
cap = release()
cv2, destroyAllwindows()
root.title("Audio and video capture").
tk.button(root, text = "Record video" & "Start Webcam")
root.mainloop()
```

RESULT:

Thus a program that captures a live video, ~~audio~~ audio from microphone and webcam has been implemented.